Draw It or Lose It

Software Design Project

Version 2.0

Table of Contents

Software Design Project 1

- **Table of Contents** 2
- **Document Revision History** 2
- **Executive Summary** 3
- **Design Constraints** 3
- **System Architecture View** 3
- **Domain Model** 3
- **Evaluation** 4
- **Recommendations** 7

Document Revision History

Version	Date	Author	Comments
1.0	07/01/2023	Jennie Mabb	Stakeholder Discovery
1.5	07/31/2023	Jennie Mabb	Initial Evaluation
2.0	08/15/2023	Jennie Mabb	Final Architectural Recommendation

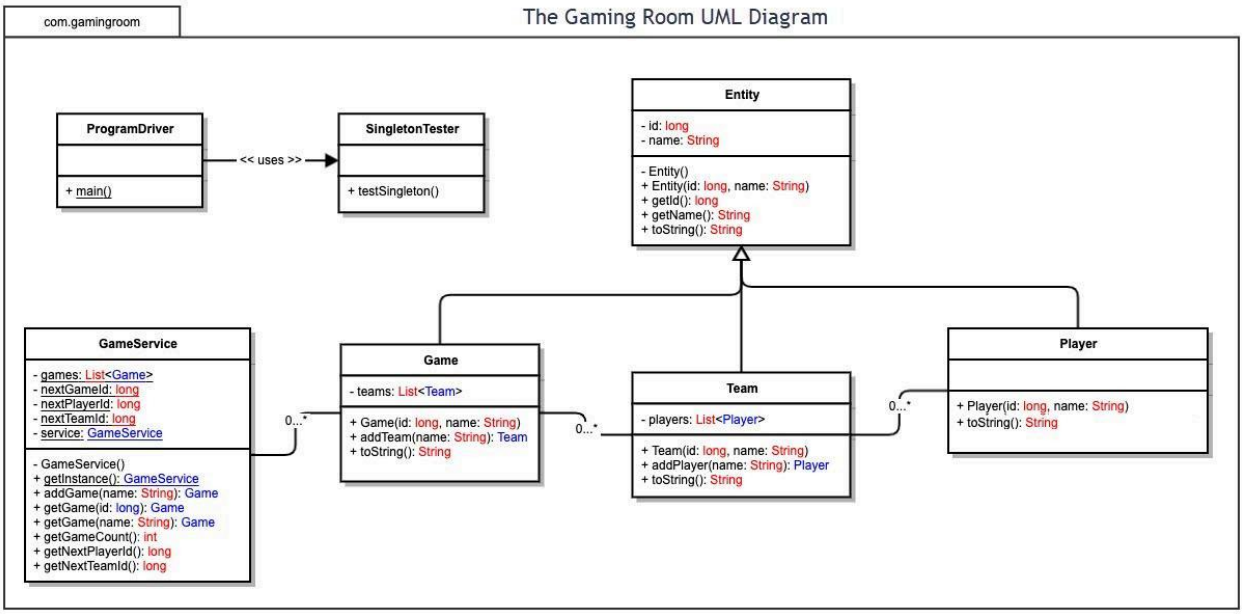
Executive Summary

The design template summary is for developing a web-based, multiplayer version of The Gaming Room's (TGR) game, Draw It or Lose It. TGR currently does not have a web presence for the Draw It or Lose It game. TGR needs application development to be inclusive of the provided software requirements. Hardware recommendations will follow later. This document version will address design constraints and the domain model.

Design Constraints

- TGR currently does not have a web-based application for Draw It or Lose It. Development of the application will need to be addressed by 3rd party, i.e., Creative Technology Solutions.
- The application design must allow for access across operating systems for customers. Customers utilize a variety of platforms to access the application, i.e., Windows-based, Mac-based, mobile-based, and Linux-based devices. Development must allow for multiplatform access.
- Development must address security features for unique game instances. TGR wants to control the game access so that a customer may have only one instance of the game loaded per unique user identification.
- Development must address security features for multi-player access with unique player names and team names. TGR's conceptual design requires that no two users share the same name, and no two teams have the same team name.
- TGR may need training and/or additional staff to support a web-based multiplatform application. TGR currently does not have staff that can attend to the development of the application and does not have Draw It or Lose It as a web application. They may not have sufficient staff to support the application once it is live, and/or have the staff knowledge to support the web-based application from a customer support standpoint.
- TGR may need infrastructure and/or additional staff to support a web-based application and its maintenance. TGR may neither have the server structure to support the application nor the staff hired/trained to maintain the application once live.

Domain Model



Evaluation - Updated for Version 1.5

Development Requirements	Mac	Linux	Windows	Mobile Devices
Server-Side	Mac servers are considered very stable as they are UNIX based. (Marijan, March 10, 2022). Unfortunately, MacOS is completely proprietary to Apple products. This leads to additional work arounds and steps for communication between non-Mac client platforms and the MacOS Server (Bartosh & Faas, 2005). Finally, MacOS requires specialized developer knowledge which the TGR may not currently have.	Linux Server OS is free and has better security in comparison to Windows Server OS or MacOS (Marijan, March 10, 2022). Despite being Open Source, Linux Server OS does require some specialized knowledge for deployment and maintenance; however, there are numerous free courses available. If staffing maintenance for Linux is a problem, there is RedHat Enterprise Linux (RHEL), which may be appropriate for a medium sized company given costs, or SUSE Linux Enterprise Server (SLES) for larger sized companies. Although these services may be costly, both of these packages allow for patches, fixes, and security updates, thus reducing the extensive knowledge for server management.	Windows Server OS has been known to be easier to use as it has a similar look and feel to the home computer screen (Marijan, March 10, 2022); which may be more familiar for current TGR staff. Additionally, Windows Server OS provides its own virtual memory management. Conversely, Windows Server OS requires licensing and is known to have a stronger risk for security threats, which results in significant costs to the TGR. Furthermore, even with additional support service agreements Windows Servers are notorious for rebooting needs, crashes, and bugs (Huffman, 2014). These deficiencies result in increased downtime to clients and could result in revenue loss through reliability concerns to TGR.	There are multiple servers that are compliant with mobile devices. Mobile devices themselves have multiple OS options, to be discussed in the Client-Side section. You can host a server on an Android device, but this is not scalable and not an option for the TGR requirements.

Client-Side	A Mac hardware client can connect to a Windows or Linux Server OS. Where the development concern arises is ensuring the game client is compatible with Safari Browser. Conversely, Draw It or Lose It could be based on Google Chrome as it is web-based platform; however, Google Chrome is only compatible for MacOS X El Capitan 10.11 or higher. Safari development tools utilize CSS, HTML, and JavaScript, plus there is a web-extension conversion tool, XCode. However, Safari development does require specialized knowledge and may result in additional staff or training requirements for TGR. An additional consideration is if Draw It or Lose It is an application that is run on a Mac system versus through a browser, then development of the program could be significantly impacted considering which language or development environment that is chosen.	Clients utilizing a Linux platform have numerous browser options available to access Draw It or Lose It as a web-based game. Development for Linux clients should not need to be different from that of Windows clients. For example, Linux clients can run Google Chrome and Microsoft Edge as their browsers for web-based games. An additional consideration is if Draw It or Lose It is an application that is run on a Linux system versus through a browser, then development of the program could be impacted considering which language or development environment that is chosen. Staffing resources may need specialized knowledge to maintain the program on the client-side.	As with clients utilizing Linux platform, there are numerous browser options available to access Draw It or Lose It as a web-based game. Development time and costs should not be impacted any differently than for a Linux client. An additional consideration is if Draw It or Lose It is an application that is run on a Windows system versus through a browser, then development of the program could be impacted considering which language or development environment that is chosen. As Windows on the client side is more common than Mac or Linux, the staffing resource constraints may only exist if specialized knowledge is required.	Mobile clients' access to Draw It or Lose It would depend on their browser capabilities. However, iPhone, Android, and Blackberry users can all access Google Chrome as their browser. Therefore, there is a common web-browser option for mobile clients. An additional consideration is if Draw It or Lose It is an application that is run on individual mobile systems versus through a browser, then development of the program could be significantly impacted considering which language or development environment that is chosen.

Development Tools	Swift is the primary development language for MacOS, iOS, and other Apple operating system products. As noted with Safari, Apple has an extension tool for coding, XCode, that also is utilized in the IDE. It is designed for multi-platform coding within the Apple suite of products. However, there are multi-platform IDEs available that include MacOS for development options. Examples include but are not limited to: Microsoft Visual Studio, NetBeans, Lazarus, Geany, and Eclipse. Some of the IDEs are available through GNU general public license; whereas others, like Microsoft Visual Studio, are proprietary software.	Linux is primarily a C programming language, with some other attributes included. There are at least 50 different IDEs available for Linux development. Examples include but are not limited to: Microsoft Visual Studio, NetBeans, Lazarus, Geany, Komodo Edit, Eclipse. Some of the IDEs are available through GNU general public license, whereas others, like Microsoft Visual Studio, are proprietary software.	Windows programing language is a blend of C, C++, and C#. There are more than 50 IDEs available for Windows with some cross-platform options. Examples include but are not limited to: Microsoft Visual Studio, NetBeans, Lazarus, Geany, Komodo Edit, Eclipse. Some of the IDEs are available through GNU general public license, whereas others, like Microsoft Visual Studio, are proprietary software.	Programming languages for mobile application development tend to align with the type of OS available (Panhale, 2016). For example, Swift is utilized for iPhones. However, Kotlin works across all platforms (MacOS, Windows, Linux, and mobile) plus allows for expansion into other platforms at a later date. There are at least ten IDEs available for Mobile development, however, most, if not all, are proprietary software. Some examples include but are not limited to: Microsoft Visual Studio, XCode, Android Studio, GeneXus. Not all these IDEs will work across all OS platforms for each mobile device, such as iPhone, Android, Blackberry (which has its own IDE).

<u>Recommendations - Updated for Version 2.0</u>

1. **Operating Platform**: Recommended Server platform is Linux, specifically, Ubuntu.

2. **Operating Systems Architectures**: Linux Server OS is open source and has better security in comparison to Windows Server OS or MacOS (Marijan, March 10, 2022). There are numerous Linux server options available; however, Ubuntu is one of the most popular and has access to WireGuard VPN for an added layer of security. Despite being Open Source, Linux Server OS does require some specialized knowledge for deployment and maintenance; however, there are numerous free courses available for enhancing Linux skills. Additionally, there are free tutorials directly from Ubuntu that have step-by-step instructions for developers (Canonical, Ltd., 2022a). Finally, Ubuntu has its own support packages available for purchase that can facilitate updates and live support staff with costs significantly less than Linux RedHat or SUSE (Canonical, Ltd., 2022b).

3. **Storage Management**: Recommended storage management is cloud storage. Cloud storage allows for infrastructure as a service and auto-scales to need (Lavieri, 2019). There are multiple options for cloud storage, and some come at a higher price point. However, cloud storage is a cheaper alternative to physical, on-site storage options. For consideration, Wasabi happens to be one of the larger, reliable alternatives to Azure, Google, and Amazon S3 (Wasabi, 2022).

   Recommendation for how storage is utilized is a blend between the end-user and the server. The end-user would have the base game installed on their device, and updates would then be on-demand from the cloud storage; thus, reducing the costs of data transfers from the server to the multitude of users accessing the game at any given point in time. The throughput of data from a cloud storage is just as fast, if not faster, than on-premises storage; therefore, there would be no I/O bottleneck of data transfer to/from storage based on the cloud location. In order to best achieve this server-client/user blend for storage, there will need to be system requirements for the game application. Minimum requirements for end-user storage for the base game should be between 8 to 16 GB, with a recommendation of 32 GB for future scalability. Given that the average mobile device has at least 32 GB, this should not prohibit a mobile user.

4. **Memory Management**: As mentioned above, the server would have on-demand transfer of data to the user. Storage of the base game would rest within the end-user's storage, with a log-on feature that checks for updates of new content at each login. Images would be indexed within the storage for appropriate retrieval. Data-oriented design for the game application allows for the user's memory and storage cache to be more organized and readily available for pulling images for each game iteration (Siblerschatz et al., 2009). New updates would be transferred from TGR's cloud storage upon login as an update. Dynamic memory allocation would allow for managing the initial download of application data as well as updates from the cloud server that would index new information accurately (Serrano, 2016; Siblerschatz et al., 2009). If capitalizing on a blend of server and end-user storage, memory management also has system requirement recommendations. At launch, the game client would run in an end-user's RAM upon launch. There needs to be a minimum RAM requirement between 8 to 16 GB, with a recommendation of 32 GB for future scalability. Again, this should not prohibit a mobile user. The option of holding all of the data within the cloud that is accessed by the end-user at each game interval through an I/O transfer is definitely a possibility that could also reduce the risk of memory indexing failures (Hennessy & Patterson, 2011; Siblerschatz et al., 2009) and could be considered as part of game application scaling in the future.

5. **Distributed Systems and Networks**: For Draw It or Lose It, the server would host a series of virtual machines from which the game client will run. These virtual machines will be connected as a network of distributed systems, which will allow ad-hoc scalability depending on the number of users accessing the server at any given point in time. The use of virtual machines will also allow users across platforms to play with each other rather than only with others utilizing the same platform. This will create load balancing for team matching, reduce bottlenecks and latency, and create a more reliable and stable experience for the end-user (Siblerschatz et al., 2009; Vijaykumar, 2009). There will be a threshold for load balancing the game and opening another VM instance to maintain the transparency of the protocol and balance the server load at any given time. This balancing protocol can be adjusted based on audit information from traffic monitoring, and historical data could pre-emptively make additional instances available. Ad-hoc scalability through distributed systems will allow users to be moved from one game instance to another for balancing user levels, instance failures, and create fault tolerance on the back-end through server and network protocols (Siblerschatz et al., 2009). Preventing bottlenecks and perceived end-user latency directly relates to

the game being deemed stable and reliable by users. Of note, a bottleneck may occur depending on the I/O of the mobile device's storage and memory or the transfer rate on WiFi or mobile data (Vijaykumar, 2009). For transparency, a pop-up should be incorporated for mobile users regarding the use of data for their service plan, as well as for response time variability between data and WiFi networks.

As **scalability** has been discussed, it is worth noting that the optimum language for application development would be Kotlin. Kotlin allows users of Windows, Mac, Android, iOS, Blackberry, etc. to be able to access the application without additional add-ons to translate the program. Furthermore, Kotlin is also scalable to other platforms in the future, such as Xbox and PlayStation. Having the application developed at the start in Kotlin would negate the need for creating bridges between platforms or having the game developed and maintained in multiple coding languages. Beginning with Kotlin will help reduce long-term costs for future releases and client platform launches.

6. **Security**:

   <u>From the server perspective:</u> Ubuntu has the capability to add WireGuard to the server as an additional layer of protection, which would protect end-user IP addresses and further data encryption. Some protocol measures would include the utilization of SSL certificates sent through JSON to encrypt user data as part of the request in a uniform manner, authentication between the server and the user, and a security policy in place for each operating system and platform, maintaining privileges and access. The mechanisms of security then dictate when a privilege is needed and thus granted, or not (Siblerchatz, et al., 2009). The concept of role-based access (RBAC) allows for the developer to ensure that each user has only the level of access needed and no more, the principle of least privilege; therefore, users will only have access to what is needed to play the game. Auditing and accountability will review the privilege access, results of vulnerability scans and intrusion detection, review performance of the servers for spikes in latency, and other intentional or accidental shifts in system reliability (Siblerchatz, et al., 2009). Routine back-ups of the server will be done in the event of a catastrophic event, and restoration needs to occur. Security scans will be done not only on the servers, but also on the virtual machines that host the games for the users. Tripwire is an example of a vulnerability security scan for servers (Siblerschatz, et al., 2009). Additionally, intrusion detection and prevention systems should be utilized on the server to address traffic security, such as Snort, which works with the recommended Ubuntu Linux Server (Siblerschatz, et al., 2009).

   <u>From the end-user perspective:</u> End-user licensing agreements will be put in place to communicate to the end-user their roles in the security of their own system, the game client, and between users. Violation of any of the requirements and stipulations would result in consequences for the end-user in accordance with any security protocol established, i.e., account freeze, banning, etc. Additionally, security mitigation will entail email reminders regarding phishing scams and that the company would not ask for password information, a security policy prompted by the server for the end-user to change their password every 30 days, with a complexity requirement. Finally, allowing the end-user the option to add authentication tokens to their account would provide an added layer of security for the end-user and the communication between the end-user and the game client. Some game companies have their own authentication token generation, similar to an RSA token, and there are also mobile applications that do this as well; for example, Microsoft has its own authenticator mobile application. A suggestion is to provide a bonus of some sort to encourage the use of the authentication layer.

7. **References:**

Bartosh, M., & Faas, R. (2005). Essential Mac OS X Panther Server Administration. O'Reilly Media, Inc.

Canonical, Ltd., (2022b). Ubuntu Advantage. Retrieved from: https://ubuntu.com/support

Canonical, Ltd., (2022a). Tutorials. Retrieved from: https://ubuntu.com/tutorials

Hennessy, J.L. and Patterson, D.A. (2011). Computer Architecture: A Quantitative Approach, 5th Ed. Morgan Kaufmann.

Huffman, C. (2014). Windows Performance Analysis Field Guide. Elsevier Inc.

Lavieri, E. (2019). Hands-On Design Patterns with Java. Packt Publishing, Ltd.

Marijan, B. (March 10, 2022). Server Operating Systems: Server OS Types & How to Choose. PhoenixNAP Global IT Services. Retrieved from: https://phoenixnap.com/kb/server-operating-system.

Panhale, M. (2016). Beginning Hybrid Mobile Application Development. Springer Science.

Siblerschatz, A., Galvin, P.B., Gagne, G. (2009). Operating System Concepts, 8th Ed. John Wiley & Sons.

Vijaykumar, T.N. (2009). Chapter 6 I/O. EE565 Lecture Notes. Purdue University Department of Engineering. https://engineering.purdue.edu/~ee565/slides/ch6.pdf.

Wasabi Technologies. (2022). Cloud Storage Pricing. Retrieved from: https://wasabi.com/cloud-storage-pricing/